



IES ALBARREGAS

Departamento de Informática

**TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA**



MANUAL TÉCNICO

Autor: Juan Eloy Ortiz Lara.

Nombre del proyecto: PadelPro creado por Club007.

Grupo: DAM 2B.

Tutor/a: María Mercedes Martínez Fragoso.

Curso académico 2025/2026

Fecha y versión: v1.0 a 4/11/2025

Enlaces importantes: TRELLO, FIGMA y GITHUB

Repositorio: <https://github.com/juane77/PadelPro-TFG>

Licencias: GPL GNU v3.

Queda por escrito que este trabajo es de carácter educativo por lo que no podrá ser utilizado para ningún otro fin sin el consentimiento del autor y del centro educativo.

Índice

1. Introducción y requisitos del sistema
2. Stack tecnológico
3. Estructura del proyecto
 - 3.1. Frontend Flutter
 - 3.2. Backend Spring Boot
4. Instalación y configuración del entorno de desarrollo
 - 4.1. Requisitos previos
 - 4.2. Configuración del backend
 - 4.3. Configuración del frontend
5. Variables de entorno y credenciales
6. Modelo de base de datos
 - 6.1. Tablas principales
 - 6.2. Relaciones entre entidades
7. API REST — Endpoints principales
8. Arquitectura de seguridad
9. Despliegue en producción
10. Diagrama de secuencia

1. Introducción y requisitos del sistema

Este manual técnico está dirigido a desarrolladores que necesiten instalar, configurar, mantener o ampliar la aplicación PadelPro. Se describe la arquitectura del sistema, la estructura del código, los pasos de instalación y los aspectos técnicos más relevantes del desarrollo.

Requisitos mínimos del sistema para ejecutar la aplicación en local:

Java	Java 21 o superior (JDK)
Maven	Maven 3.8 o superior
Flutter	Flutter 3.38.5 / Dart 3.10.4
Node.js	Node.js 18 o superior (para build web)
PostgreSQL	PostgreSQL 15 o acceso a Supabase
IDE recomendado	IntelliJ IDEA para backend, VS Code o Android Studio para Flutter
Sistema operativo	Windows 10/11, macOS 12+ o Linux Ubuntu 20.04+

2. Stack tecnológico

Flutter 3.38.5 / Dart 3.10.4	Framework multiplataforma para Android y Web
Java 21 + Spring Boot 4.0.2	Backend con API REST
Hibernate 7 / Spring Data JPA	ORM para acceso a base de datos
PostgreSQL 15	Base de datos relacional
Supabase	PostgreSQL gestionado en la nube + Storage
JWT (JSON Web Tokens)	Autenticación sin estado
BCrypt	Cifrado de contraseñas
Brevo API HTTP	Envío de emails transaccionales
Supabase Storage	Almacenamiento de fotos de perfil
Render	Hosting del backend con Docker
Vercel	Hosting de la versión web de Flutter
Cloudflare Workers	Hosting de la landing page
GitHub	Control de versiones y CI/CD

3. Estructura del proyecto

3.1. Frontend Flutter

El frontend sigue una estructura de carpetas organizada por tipo de archivo:

```
lib/  
  main.dart                # Punto de entrada  
  models/                  # Clases de modelo (Pista, etc.)  
  screens/                  # Pantallas de la app (19 screens)  
    login_screen.dart  
    register_screen.dart  
    home_screen.dart  
    matches_screen.dart  
    news_screen.dart  
    profile_screen.dart  
    reservation_calendar_screen.dart  
    mis_reservas_screen.dart  
    amigos_screen.dart  
    ...  
  service/                  # Servicios de acceso a datos  
    api_service.dart        # Gestión de pistas  
    reserva_service.dart  
    partido_service.dart  
    amistad_service.dart  
    foto_service.dart       # Subida a Supabase Storage  
    session.dart            # Estado global del usuario  
    reserva_provider.dart   # Provider para reservas  
  utils/                    # Utilidades (responsive, snackbar)  
  widgets/                  # Widgets reutilizables (AppBar)  
assets/  
  images/                   # Imágenes y logos  
  fonts/                    # Fuentes (Poppins)
```

3.2. Backend Spring Boot

```
src/main/java/com/tfg/padelpro/  
  PadelproApplication.java  # Clase principal  
  controller/               # Controladores REST  
    UsuarioController.java  
    ReservaController.java  
    PartidoController.java  
    AmistadController.java  
    AdminController.java  
    PistaController.java  
    ClubController.java  
  model/                     # Entidades JPA  
  repository/                # Repositorios Spring Data  
  service/                   # Lógica de negocio  
  security/                  # JWT filter, SecurityConfig  
  scheduler/                 # ReservaScheduler (tarea diaria)  
src/main/resources/  
  application.properties    # Configuración (variables de entorno)  
  static/admin.html         # Panel de administración web
```

4. Instalación y configuración del entorno de desarrollo

4.1. Requisitos previos

Antes de comenzar, asegúrate de tener instalados los siguientes componentes:

- Java 21 JDK (<https://adoptium.net>)
- Maven 3.8+ (incluido en IntelliJ IDEA)
- Flutter SDK 3.38.5 (<https://flutter.dev/docs/get-started/install>)
- Git (<https://git-scm.com>)
- Android Studio o VS Code con extensión Flutter

4.2. Configuración del backend

1. Clona el repositorio:

```
git clone https://github.com/juane77/PadelPro-TFG.git
cd PadelPro-TFG/backend/padelpro/padelpro
```

2. Crea un archivo .env o configura las variables de entorno del sistema (ver sección 5).

3. Ejecuta el backend:

```
mvn spring-boot:run
```

El backend estará disponible en <http://localhost:8080>.

4.3. Configuración del frontend

1. Accede a la carpeta del frontend:

```
cd PadelPro-TFG/frontend/PadelPro/PadelPro
```

2. Instala las dependencias:

```
flutter pub get
```

3. Ejecuta en Android (con dispositivo o emulador conectado):

```
flutter run
```

4. Ejecuta en web:

```
flutter run -d chrome
```

5. Genera la APK de producción:

```
flutter build apk --release
```

6. Genera la versión web de producción:

5. Variables de entorno y credenciales

El backend lee las siguientes variables de entorno en el archivo `application.properties`. En producción (Render) se configuran como variables de entorno del servicio:

DB_URL	URL JDBC de conexión a PostgreSQL. Ejemplo: <code>jdbc:postgresql://host:5432/postgres</code>
DB_USERNAME	Usuario de la base de datos (por defecto: <code>postgres</code>)
DB_PASSWORD	Contraseña de la base de datos de Supabase
JWT_SECRET	Clave secreta para firmar los tokens JWT (mínimo 32 caracteres)
BREVO_API_KEY	Clave de API de Brevo para el envío de emails

En el frontend, las URLs del backend y de Supabase están definidas en los archivos de servicio:

- URL del backend: `https://padelpro-tfg.onrender.com` (configurable en cada service)
- URL de Supabase: `https://vketxcsgjfpbgrgxwopv.supabase.co`
- Publishable key de Supabase:
`sb_publishable_nMfQPmPgELN51LgEEhtGOQ_nH1LmIOE`

6. Modelo de base de datos

6.1. Tablas principales

Tabla	Campos clave	Descripción
usuario	id, nombre, email, password, rol, pelotas, emailVerificado, ultimoLogin	Usuarios registrados en la app
club	id, nombre, ciudad, latitud, longitud	Clubs de pádel disponibles
pista	id, nombre, ciudad, precioHora, latitud, longitud, club_id	Pistas de cada club
reserva	id, usuario_id, pista_id, fechaReserva, estado (ACTIVA/CANCELADA)	Reservas de pistas
partido	id, usuario_id, pista_id, reserva_id, resultado, nivelMedio, resultadoFinal, fechaPartido	Partidos registrados
amistad	id, usuario1_id, usuario2_id, estado	Relaciones de amistad entre usuarios
solicitud_amistad	id, emisor_id, receptor_id, estado	Solicitudes de amistad pendientes
notificacion	id, usuario_id, mensaje, leida, fechaCreacion	Notificaciones del sistema

6.2. Relaciones entre entidades

- Un Club tiene muchas Pistas (1:N)
- Una Pista tiene muchas Reservas (1:N)
- Un Usuario tiene muchas Reservas (1:N)
- Una Reserva puede tener un Partido asociado (1:1)
- Un Usuario tiene muchos Partidos (1:N)
- Dos Usuarios pueden tener una Amistad (N:M gestionado con tabla intermedia)
- Un Usuario tiene muchas Notificaciones (1:N)

7. API REST — Endpoints principales

Método	Endpoint	Descripción
POST	/api/usuarios/login	Inicio de sesión. Devuelve JWT.
POST	/api/usuarios/register	Registro de nuevo usuario.
POST	/api/usuarios/confirmar-email	Verificación del código de email.
POST	/api/usuarios/recuperar-password	Solicitud de código de recuperación.
GET	/api/pistas	Lista todas las pistas disponibles.
GET	/api/reservas/usuario/{id}	Reservas de un usuario (formato plano).
POST	/api/reservas	Crear nueva reserva (-15 pelotas).
PUT	/api/reservas/{id}/cancelar	Cancelar reserva (+10 pelotas).
GET	/api/partidos/usuario/{id}	Todos los partidos de un usuario.
POST	/api/partidos	Registrar nuevo partido.
GET	/api/amistades/{id}	Lista de amigos de un usuario.
POST	/api/amistades/solicitud	Enviar solicitud de amistad.
PUT	/api/amistades/{id}/aceptar	Aceptar solicitud de amistad.
GET	/api/notificaciones/{id}	Notificaciones de un usuario.
GET	/api/admin/usuarios	Lista todos los usuarios (ADMIN).
GET	/api/admin/reservas	Lista todas las reservas (ADMIN).
PUT	/api/admin/reservas/{id}/cancelar	Cancelar reserva desde panel admin.

8. Arquitectura de seguridad

La seguridad de PadelPro se basa en los siguientes mecanismos:

Autenticación JWT

Al iniciar sesión, el backend genera un token JWT firmado con una clave secreta configurada como variable de entorno. El token tiene una validez de 24 horas. El frontend incluye este token en la cabecera `Authorization: Bearer {token}` en todas las peticiones autenticadas.

Cifrado de contraseñas

Las contraseñas se almacenan cifradas con BCrypt (factor de coste 10). Nunca se almacenan en texto plano ni se devuelven en las respuestas de la API.

Control de roles

La aplicación dispone de tres roles: USER (jugador estándar), ADMIN (administrador de club) y SUPERADMIN (acceso total). Los endpoints del panel de administración están protegidos mediante anotaciones `@PreAuthorize` en los controladores.

Verificación de email

Al registrarse, el usuario recibe un código de 6 dígitos por email con validez de 15 minutos. La cuenta permanece bloqueada hasta que el email es verificado.

CORS

El backend está configurado para aceptar peticiones desde los orígenes de la versión web (Vercel) y desde localhost para desarrollo. La configuración se encuentra en `SecurityConfig.java`.

9. Despliegue en producción

Despliegue del backend en Render

- El repositorio de GitHub está conectado a Render con despliegue automático en cada push a main.
- Render utiliza el Dockerfile del proyecto para construir la imagen.
- Las variables de entorno (DB_PASSWORD, BREVO_API_KEY, JWT_SECRET) se configuran en el panel de Render.
- URL de producción: <https://padelpro-tfg.onrender.com>

Despliegue de la web en Vercel

- El repositorio está conectado a Vercel con despliegue automático.
- El directorio de publicación es build/web (resultado de flutter build web --release).
- URL de producción: <https://padel-pro-tfg.vercel.app>

Despliegue manual de la APK

- Generar con: flutter build apk --release
- La APK se encuentra en: build/app/outputs/flutter-apk/app-release.apk
- Se sube a Google Drive y el enlace se actualiza en la landing page.

10. Diagrama de secuencia

El siguiente diagrama simplificado muestra el flujo de datos en las operaciones principales de la aplicación:

Usuario → Flutter App → API REST (Spring Boot) → PostgreSQL (Supabase)

1. El usuario introduce sus credenciales en el login de Flutter.
2. Flutter envía POST /api/usuarios/login al backend de Spring Boot.
3. Spring Boot valida las credenciales contra PostgreSQL, genera el JWT y lo devuelve.
4. Flutter almacena el JWT en Session.token y redirige a la pantalla de inicio.
5. Todas las peticiones posteriores incluyen el JWT en la cabecera Authorization.
6. Spring Boot valida el JWT en cada petición mediante el filtro JwtAuthFilter.
7. Spring Boot consulta o modifica los datos en PostgreSQL y devuelve la respuesta en formato JSON.
8. Flutter parsea el JSON y actualiza la interfaz de usuario.